```
import akka.actor.typed.scaladsl.{ PoolRouter, Routers }

object Manager {

  def apply(behavior: Behavior[String]) =
    Behaviors.setup[Unit] { context =>
      val routingBehavior: Behavior[String] =
            Routers.pool(poolSize = 4)(behavior)          ❶
      val router: ActorRef[String] =
            context.spawn(routingBehavior, "test-pool")   ❷

    (0 to 10).foreach { n =>
      router! "hi"                                        ❸
    }
}
```

❶ Sets the pool size and the behavior to instantiate

❷ Spawns the pool

❸ Sends messages to the router and thus to its routees

This example is tested in `PoolRoutersSpec.scala` in a test called “send messages in a round-robin fashion,” if you want to play around with it.

Remember that the router can send a message to a child that has stopped. When a router’s child stops, a `Terminated(child)` message is sent to its router. But before this signal arrives, the router may already be sending a message to that child. In this case, the message is forwarded to `deadLetters`.

When the pool router is terminated, all routees are also terminated due to the parent-child relationship. Conversely, when all the children stop, the router also stops.

It is good practice to provide a restart strategy for the routees used by a pool router. When a routee fails, it is stopped by default, which